

1 Handling missing values in time series data

Missing values are common in real-world datasets due to sensor failures, data entry errors, transmission interruptions, or incomplete observations. In time series analysis, missing values can distort patterns, bias statistical estimates, and reduce the reliability of forecasts. Therefore, identifying and treating missing observations is an essential step in data preprocessing before modeling or visualization. In R, missing values are typically represented by NA.

1.1 Air Quality dataset

The `airquality` dataset in R contains 153 daily air pollution observations recorded in New York City from May to September 1973 (We will date it from 1st May 1973 to 30th September 1973). Variables include ozone concentration, solar radiation, wind speed, temperature, month, and day. Some ozone and solar radiation measurements are missing, making the dataset useful for practicing data cleaning and time-series preparation.

1.2 Identifying missing values

Before applying any correction technique, it is important to detect and quantify missing values. Common R functions include:

- `is.na()` — identifies missing values
- `sum(is.na(x))` — counts missing values
- `which(is.na(x))` — identifies the positions of missing observations

Example:

```
#confirm the number of missing values
sum(is.na(airquality$Ozone))
```

1.3 Handling missing values

Understanding the distribution of missing values helps determine whether simple imputation methods are sufficient or more sophisticated approaches are required.

1.3.1 Interpolation

Interpolation estimates missing values using nearby data points. It works well when missing observations occur within the series and the data changes smoothly over time. In R, linear interpolation can be done using `na.approx()` from the `zoo` package:

```
#install and call the zoo library
library(zoo)
#interpolate to fill in the missing values
ozone_interp <- na.approx(airquality$Ozone)
```

1.3.2 Forward Filling

Forward filling replaces missing values with the most recent available observation. It is useful when values tend to persist for short periods. In R, forward filling is implemented with `na.locf()`:

```
library(zoo)
#Forward filling method to replace missing values
ozone_ffill <- na.locf(airquality$Ozone)
```

1.3.3 Backward Filling

Backward filling replaces missing values with the next available observation:

```
ozone_bfill <- na.locf(airquality$Ozone, fromLast = TRUE)
```

1.3.4 Choosing an appropriate method

- Interpolation is suitable for continuous variables with gradual changes.
- Forward filling is appropriate when the last observation remains valid.
- Backward filling is useful for missing values at the beginning of a series.

If a large fraction of data is missing, more advanced techniques or dataset reconstruction may be necessary.

1.4 Time indexing in time series data

Time indexing organizes observations according to their timestamps. Proper indexing ensures that temporal relationships between observations are preserved, which is essential for analysis, visualization, and forecasting. R provides several structures for time series data: `ts`, `zoo`, and `xts`.

1.4.1 The ts Object in R

The `ts` object is suitable for regularly spaced time series data. Using the built-in `airquality` dataset, which contains daily observations from May 1, 1973 to September 30, 1973, we can create a `ts` object for the Ozone variable.

Since the data are daily, we approximate the frequency as 30 observations per month. The `ts` object in R does not store actual calendar dates; it only stores a numeric time index based on the start and frequency.

```
# Create ts object for Ozone
ozone_ts <- ts(airquality$Ozone,
start = c(1973, 5),
frequency = 30)
```

Advantages:

- Simple structure for regular series
- Compatible with forecasting and decomposition functions
- Built-in plotting for time series visualization

1.4.2 The zoo Package

The `zoo` package can handle irregular and regular time series. Here we create a `zoo` object for the Ozone variable indexed by actual dates from May 1 to September 30, 1973.

```
library(zoo)
# Create date sequence
dates <- seq(as.Date("1973-05-01"), as.Date("1973-09-30"), by = "day")
# Create zoo object (ensure lengths match)
ozone_zoo <- zoo(airquality$Ozone, dates)
```

Advantages:

- Flexible indexing using real dates
- Handles missing values with interpolation functions
- Suitable for irregularly spaced time series

1.4.3 The xts Package

The `xts` package builds upon `zoo` and is optimized for high-performance operations. It allows subsetting, aggregation, and plotting efficiently. We can create an `xts` object using the same `airquality` data:

```
library(xts)
# Create xts object
ozone_xts <- xts(airquality$Ozone, order.by = dates)
```

Features:

- Fast indexing and subsetting by date ranges
- Easy aggregation (daily $\rightarrow$ monthly averages)
- Compatible with time-based plotting and financial/time series tools

1.5 Importance of proper time indexing

- Preserves chronological order
- Supports temporal operations like resampling and smoothing
- Improves visualization of time trends
- Facilitates accurate forecasting

Handling missing values and properly indexing time series are fundamental steps in preprocessing. Missing values can be treated via interpolation, forward filling, or backward filling. Time indexing structures, such as `ts`, `zoo`, and `xts`, allow analysts to organize data chronologically for accurate analysis, visualization, and forecasting.

2 Introduction to outliers

Outliers are observations that differ markedly from the rest of the data. They may occur due to measurement errors, recording mistakes, or natural variability. Detecting outliers is critical in time series analysis because they can bias estimates, distort trends, and affect forecasting.

The `airquality` dataset contains daily air quality measurements in New York from May to September 1973, including variables Ozone, Solar.R, Wind, Temp, Month, and Day. It is ideal for practicing outlier detection.

2.1 Main methods to detect outliers

We focus on three primary methods for detecting outliers in R: Boxplots, Z-score, and Interquartile Range (IQR). Other methods such as `tsoutliers()` for time series or Mahalanobis distance for multivariate data are mentioned briefly.

2.2 Boxplot method

Boxplots identify observations outside 1.5 times the interquartile range (IQR) as potential outliers.

```
# Boxplot for Ozone
boxplot(airquality$Ozone,
main = "Boxplot of Ozone",
ylab = "Ozone (ppb)",
col = "lightblue",
na.action = na.omit)
```

Points beyond the whiskers are considered potential outliers. Boxplots provide a quick visual indication.

2.3 Z-Score method

The Z-score standardizes each observation. Values with absolute Z-scores greater than 3 are often flagged as outliers.

```
# Z-score method for Ozone
ozone_z <- scale(airquality$Ozone)
outliers_z <- which(abs(ozone_z) > 3)
outliers_z
# Inspect the outlier values
airquality$Ozone[outliers_z]
```

This method works best for approximately normally distributed variables.

2.4 Interquartile range (IQR) method

The IQR method uses the spread of the middle 50% of the data to identify extreme observations.

```
# IQR method for Ozone
Q1 <- quantile(airquality$Ozone, 0.25, na.rm = TRUE)
Q3 <- quantile(airquality$Ozone, 0.75, na.rm = TRUE)
IQR_val <- Q3 - Q1
lower_bound <- Q1 - 1.5 * IQR_val
upper_bound <- Q3 + 1.5 * IQR_val
outliers_iqr <- which(airquality$Ozone < lower_bound | airquality$Ozone > upper_bound)
outliers_iqr
# Display the outlier values
airquality$Ozone[outliers_iqr]
```

This method is robust to skewed distributions.

3 Other Outlier detection methods

Additional approaches include:

- Time series-specific: `tsoutliers()` in the `forecast` package.
- Robust multivariate methods: Mahalanobis distance.
- Transformations: log or Box-Cox to reduce the effect of extreme values.

In summary:

- Outliers are extreme or unusual observations that can bias analysis.
- The three primary methods in R are:
 1. Boxplot Method – visual and intuitive.
 2. Z-score Method – standardized approach for normal variables.
 3. IQR Method – robust for skewed data.
- Other methods include time series-specific approaches and multivariate distances.
- Detecting outliers is the first step; correction or treatment (removal, interpolation, or Winsorization) should follow depending on the analysis.

3.1 Methods to remove outliers

While detecting outliers is important, there are situations where it is necessary to remove them, especially if they represent measurement errors or anomalies that could bias statistical analysis.

3.1.1 IQR-based removal

The Interquartile Range (IQR) method identifies outliers based on the spread of the middle 50% of the data. Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ are considered extreme. This method is robust to skewed distributions.

```
# IQR method for Ozone variable
data("airquality")
# Compute Q1, Q3, and IQR
Q1 <- quantile(airquality$Ozone, 0.25, na.rm = TRUE)
Q3 <- quantile(airquality$Ozone, 0.75, na.rm = TRUE)
IQR_val <- Q3 - Q1
# Define acceptable range
lower_bound <- Q1 - 1.5 * IQR_val
upper_bound <- Q3 + 1.5 * IQR_val
# Identify outliers
outliers_iqr <- which(airquality$Ozone < lower_bound | airquality$Ozone > upper_bound)
# Remove outliers
airquality_iqr_clean <- airquality[-outliers_iqr, ]
# Inspect cleaned data
summary(airquality_iqr_clean$Ozone)
```

3.1.2 Z-Score-based removal

The Z-score method standardizes observations relative to the mean and standard deviation. Observations with absolute Z-scores greater than a threshold (commonly 3) are considered outliers. This method works best for approximately normal variables.

```
# Z-score method for Ozone
ozone_z <- scale(airquality$Ozone)
# Standardize Ozone values
# Identify outliers
outliers_z <- which(abs(ozone_z) > 3)
# Remove outliers
airquality_z_clean <- airquality[-outliers_z, ]
# Inspect cleaned data
summary(airquality_z_clean$Ozone)
```

In summary:

- IQR-based removal identifies outliers relative to the interquartile range and is robust to skewed data.
- Z-score-based removal removes values far from the mean in terms of standard deviations, ideal for approximately normal data.
- Both methods reduce the influence of extreme observations, improving statistical modeling, visualization, and forecasting.

4 Data smoothing and preparing datasets

Smoothing and preprocessing time series data is essential to highlight trends, reduce noise, and prepare the data for modeling and visualization. The airquality dataset contains daily measurements of Ozone, Solar.R, Wind, Temp, and dates from May to September 1973, making it ideal for applying these techniques.

4.1 Data smoothing techniques

Data smoothing reduces random fluctuations while preserving underlying trends. The two main methods are:

4.1.1 Moving averages

Moving averages smooth time series data by calculating the average of a fixed number of consecutive observations. This technique is effective for reducing short-term fluctuations while highlighting longer-term trends. The degree of smoothing is determined by the window size, with larger windows producing smoother series but potentially masking short-term changes.

```
library(zoo)
# Create a time series for Ozone
ozone_ts <- ts(airquality$Ozone, start = c(1973,5), frequency = 30)
# 7-day moving average
ozone_ma7 <- rollmean(ozone_ts, k = 7, fill = NA)
# Plot
plot(ozone_ts, type = "l", col = "gray", ylab = "Ozone", main = "7-day Moving Av
lines(ozone_ma7, col = "blue", lwd = 2)
```


4.1.2 Simple exponential smoothing (SES)

Simple exponential smoothing assigns exponentially decreasing weights to past observations, giving more importance to recent values. This allows the method to respond more quickly to recent changes while still incorporating historical data. The smoothing parameter, α , controls the responsiveness: values of α close to 1 make the series more reactive to recent changes, whereas values near 0 produce a smoother trend by giving greater weight to older observations.

```
library(forecast)
# Simple exponential smoothing
ozone_ses <- ses(ozone_ts, alpha = 0.3, initial = "simple", h = 0)
# Plot
plot(ozone_ses, main = "Simple Exponential Smoothing of Ozone", ylab = "Ozone")
```

4.2 Preparing datasets for modeling and visualization

Proper structuring and cleaning of time series data is essential before modeling or visualization. Two main techniques are time indexing and handling missing values. These methods have been covered in the earlier sections of this module. Namely, time indexing and handling missing values.

5 Practical Application

The R code shows all the steps we have considered in this module.

```
# Load required packages
library(zoo)
library(xts)
library(forecast)
# Load airquality dataset
data("airquality")
# Create zoo object with proper dates
dates <- seq(as.Date("1973-05-01"), as.Date("1973-09-30"), by = "day")
ozone_zoo <- zoo(airquality$Ozone, order.by = dates)
# Linear interpolation
ozone_interp <- na.approx(ozone_zoo)
# Forward fill (alternative)
ozone_ffill <- na.locf(ozone_zoo)
# IQR method
```

```

Q1 <- quantile(ozone_interp, 0.25, na.rm = TRUE)
Q3 <- quantile(ozone_interp, 0.75, na.rm = TRUE)
IQR_val <- Q3 - Q1
lower <- Q1 - 1.5 * IQR_val
upper <- Q3 + 1.5 * IQR_val
# Remove outliers
ozone_clean <- ozone_interp[ozone_interp >= lower & ozone_interp <= upper]
# Create xts object for easier plotting and aggregation
ozone_xts <- xts(ozone_clean, order.by = index(ozone_clean))
# Moving average (7-day)
ozone_ma7 <- rollmean(ozone_xts, k = 7, fill = NA)
# Simple exponential smoothing
ozone_ses <- ses(as.numeric(ozone_xts), alpha = 0.3, initial = "simple", h = 1)
# Plot original, moving average, and SES
plot(index(ozone_xts), coredata(ozone_xts), type = "l", col = "gray",
ylab = "Ozone (ppb)", main = "Ozone Concentration with Smoothing")
lines(index(ozone_ma7), coredata(ozone_ma7), col = "blue", lwd = 2)
lines(index(ozone_xts), fitted(ozone_ses), col = "red", lwd = 2)
legend("topright", legend = c("Original", "7-day MA", "SES"),
col = c("gray", "blue", "red"), lwd = 2)

```